CogniLens Major Product Updates and AI-Assisted Programming Prompts

Based on GitHub commit history

This document summarises the major product-level updates in CogniLens and reconstructs detailed Cursor-style programming prompts for each phase. The prompts are written as implementation briefs that reflect the project context, code boundaries, validation expectations, and acceptance criteria visible from the GitHub submission history.Evidence note: The commit hashes below are representative evidence from the repository history. The prompts are reconstructed as AI-assisted implementation briefs aligned with the actual development phases, not as a claim that every line was submitted verbatim to an AI tool.

1. MVP foundation and first cognitive accessibility analyzers

Commit evidence: 9cd9330, e82776f, b00d9e5, 80302b9, d254c4a, 310bc6b

Representative contributors: Henrrryuan, YanyaL, runhong, gldwen

What changed

The project moved from planning into a working CogniLens MVP with a FastAPI backend, static frontend, basic schemas, sample pages, and the first cognitive accessibility analyzer rules.Set up the public project structure and the first backend/frontend workflow.Added early readability, visual/information overload, interaction, and consistency analyzers.Introduced test database planning and sample pages so later history and reporting features could be grounded.

AI-assisted programming prompt

Implementation prompt

You are working in the CogniLens repository. Build the first working MVP for a cognitive accessibility assistant.

Current context:
- The project is moving from concept into implementation.
- Existing code may contain early static pages, prototype scripts, and sample files.
- CogniLens should analyse webpages and return cognitive accessibility findings for web developers.
- Do not over-engineer the architecture, but keep the structure ready for future dashboard and history features.

Goal:
Create a working MVP foundation with backend analysis, frontend entry points, basic result schemas, and first rule-based analyzers.

Part 1: Backend foundation
- Set up or align FastAPI routes for analysing HTML input.
- Define structured response fields for dimensions, issues, evidence, and scoring.
- Add safe error handling for missing or invalid HTML.

Implementation prompt (continued)

Part 2: First analyzer modules
- Implement early detectors for readability, visual/information overload, interaction/distraction, and consistency.
- Each rule should return an id, title, description, evidence, suggestion, and affected element data where possible.
- Keep detector logic deterministic and explainable.

Part 3: Frontend MVP

- Connect a simple frontend flow to the backend analysis API.
- Show analysis results in a readable prototype dashboard.
- Preserve sample pages for repeatable testing.

Validation:
- Run the backend locally and analyse at least one sample HTML page.
- Confirm the frontend can trigger analysis and display returned findings.
- Confirm missing fields do not crash the UI.

Implementation prompt (continued)

Acceptance criteria:
- CogniLens has a working analyse-and-display loop.
- The first analyzers return structured cognitive accessibility findings.
- The output is understandable enough for later dashboard issue cards.

After implementation, summarize:
- files changed
- routes and schemas added
- analyzer rules implemented
- sample validation results

2. Upload workflow, report history, comparison, search, and print support

Commit evidence: 48aef05, 8296307, feb71be, a8554ec, 7a93e5b

Representative contributors: YanyaL, gldwen

What changed

CogniLens became more than a one-off analyser. The product gained upload support, stored analysis history, report comparison, searchable history, and print/export behaviour.Added HTML/ZIP upload paths and dependency checks for multi-file website packages.Introduced SQLite-backed report history with analysis runs and comparison flow.Improved the History page with search and print actions for reviewing previous reports.

AI-assisted programming prompt

Implementation prompt

Extend CogniLens from a single-page analyzer into a report-based workflow.

Current context:
- Users need to analyse local webpages and return to previous reports later.
- Existing analysis output should be stored instead of disappearing after one run.
- ZIP upload support is needed because real static sites often contain separate HTML, CSS, JS, and assets.
- Do not remove the existing single HTML analysis path.

Goal:
Add upload, history, comparison, search, and print support so CogniLens can support repeatable report review.

Part 1: Upload workflow
- Add support for HTML upload and ZIP upload where available.
- Validate file types, empty archives, and missing HTML entry files.

- Preserve relative asset paths for preview where possible.

Implementation prompt (continued)

Part 2: Persistent report history
- Store each analysis run with source name, timestamp, HTML snapshot, dimension results, issues, evidence JSON, and DOM location JSON.
- Add routes to list history, fetch one report, and compare reports where supported.
- Keep storage compatible with future evidence fields.

Part 3: History UI
- Add a History page that lists previous analysis runs.
- Add search or filtering by report name where practical.
- Add print support without breaking dashboard navigation.

Validation:
- Upload a simple HTML file and confirm it is analysed and saved.
- Upload a ZIP file and confirm validation messages are clear.
- Reopen a saved report and confirm the result matches the original run.

Implementation prompt (continued)

Acceptance criteria:
- Users can upload, analyse, save, search, reopen, compare, and print reports.
- History data is persisted in SQLite.
- Existing analysis routes still work.

After implementation, summarize:
- files changed
- upload paths added
- history schema/storage changes
- UI actions added
- validation results

3. Unified FastAPI app and eye-tracking evidence layer

Commit evidence: fc5e0a9, ad9bec7, 0c5da99, b500ecd, 50ac4bd

Representative contributors: YanyaL

What changed

The separate eye-tracking prototype was connected to the main CogniLens product as an optional behavioural validation layer, with larger preview layout and history-linked session storage.Unified CogniLens and the eye-tracking page under one FastAPI app.Added proxy/preview support and refined the eye-tracking layout around the tested webpage.Linked eye sessions to report history so behavioural evidence could be reopened later.

AI-assisted programming prompt

Implementation prompt

Integrate the existing eye-tracking prototype into CogniLens as an optional behavioural evidence layer.

Current context:
- CogniLens already has static cognitive accessibility analysis and report history.
- A separate eye-tracking prototype exists, but it is not yet part of the main workflow.
- The tested webpage should be visible inside the eye-tracking experience.
- Do not replace the main static analysis workflow.

Goal:
Serve the eye-tracking workflow through the same FastAPI product and connect sessions to saved analysis history.

Part 1: App integration
- Serve the eye-tracking page from the main FastAPI app.
- Add proxy support if needed so a target webpage can be loaded inside the product flow.
- Keep static assets and API routes clearly separated.

Implementation prompt (continued)

Part 2: Eye session storage
- Store session metadata such as run id, source name, target URL, sample count, duration, coverage, and heatmap data.
- Link eye sessions to analysis reports when a run id is available.
- Preserve backwards compatibility for reports without eye evidence.

Part 3: Eye UI layout
- Increase the tested-page preview area so users can observe the page naturally.
- Keep controls visible without blocking the target webpage.
- Rename evidence labels so they communicate behavioural support clearly.

Validation:
- Open the eye-tracking page through the main backend server.
- Load a target webpage in the preview area.
- Save an eye session and confirm it appears in history-linked data.

Implementation prompt (continued)

Acceptance criteria:
- Eye tracking is part of the CogniLens app, not a separate tool.
- Sessions can be stored and linked to report history.
- Reports without eye evidence continue to work.

After implementation, summarize:
- files changed
- routes added
- storage fields added
- layout changes
- manual test results

4. Rule mapping, scoring redesign, issue guidance, and backend layering

Commit evidence: 70eeaf0, 248b9fd, f2202f6, 2704673, 7526141, 72eaa9c

Representative contributors: Henrrryuan, runhong, gldwen

What changed

The system shifted from raw detector output toward clearer cognitive accessibility guidance. Backend modules were separated and issue cards gained standards mapping and risk-oriented language.Refactored backend ownership into app, routers, services, adapters, analyzers, and schemas.Mapped rules to WCAG/COGA-style guidance and displayed standards directly in results.Replaced abstract category scores with risk badges and clearer issue guidance.

AI-assisted programming prompt

Implementation prompt

Refactor CogniLens so backend ownership is clearer and dashboard findings are easier to interpret.

Current context:
- The analysis engine has grown beyond the first prototype.
- Detector outputs need clearer mapping to cognitive accessibility principles and standards.
- Dashboard users need risk-oriented language rather than abstract scores only.
- Preserve existing API behaviour where possible.

Goal:
Improve backend layering, rule metadata, scoring language, and issue guidance without breaking the user workflow.

Part 1: Backend layering
- Separate route handlers, service orchestration, persistence adapters, input adapters, analyzers, and schemas.
- Keep module names aligned with detector responsibility.
- Avoid broad rewrites outside the analysis path.

Implementation prompt (continued)

Part 2: Rule metadata and standards mapping
- Add or refine rule metadata for each detector.
- Include relevant cognitive accessibility guidance, WCAG/COGA-style references, or ISO usability principles where appropriate.
- Make mapping visible in result data so the frontend can display it.

Part 3: Scoring and risk language
- Reduce over-reliance on abstract category scores.
- Introduce Low / Medium / High risk badges where useful.
- Keep scoring deterministic and explainable.

Part 4: Issue guidance
- Improve user-facing explanations and recommendations.
- Make issue cards answer: what was detected, why it matters, and what to change.

Implementation prompt (continued)

Validation:
- Run sample analyses before and after the refactor.
- Confirm result shapes remain compatible.
- Confirm dashboard issue cards still render.

Acceptance criteria:
- Backend modules have clearer ownership boundaries.
- Rules include standards-oriented metadata.

- Dashboard risk language is more actionable.
- Existing routes and analysis workflows still work.

After implementation, summarize:
- files changed
- backend modules reorganised
- rule metadata added
- scoring/risk display changes
- validation results

5. React/Vite migration and accessibility widget expansion

Commit evidence: 2ee0a78, c6124ea, b7b5e11, b5cbd34, 5cc1d8a, 54f4d06, 1c98343, df01755, 29ee9e2, 82cf53f, ca1f71b

Representative contributors: YanyaL, runhong, gldwen

What changed

The frontend moved into a Vite + React architecture while preserving legacy dashboard behaviour. At the same time, the accessibility widget expanded with profiles and reading support tools.Migrated homepage, loading, dashboard, history, and eye workflow into React routes and components.Fixed React history routing and local development proxy behaviour.Expanded accessibility tools with reading aid, readable fonts, text reader, saturation, page structure view, and cognitive profiles.

AI-assisted programming prompt

Implementation prompt

Migrate the CogniLens frontend from static HTML/JS into a Vite + React application while preserving the existing workflow.

Current context:
- CogniLens has working static frontend pages and legacy dashboard scripts.
- The project now needs a maintainable React structure for continued development.
- Important pages include homepage, loading, dashboard, history, docs, and eye workflow entry points.
- Complex dashboard logic can remain as legacy modules during migration.

Goal:
Create a React/Vite frontend that keeps the product functional while making page structure, routing, and future UI changes easier to manage.

Part 1: React architecture
- Set up Vite, React entry point, React Router, shared layout, and reusable page components.
- Add routes for homepage, loading, dashboard, history, docs, and eye-related navigation.
- Move shared API helpers into a frontend utility module.

Implementation prompt (continued)

Part 2: Legacy compatibility
- Preserve complex dashboard behaviour by wrapping or importing legacy modules where needed.
- Avoid duplicating old and new logic.
- Keep API contracts unchanged.

Part 3: Development proxy and routing
- Configure local frontend dev server proxy to the backend.

- Resolve conflicts between React history routes and API paths.
- Avoid 404s on page refresh.

Part 4: Accessibility widget expansion
- Integrate or preserve floating accessibility menu features.
- Support reading masks, readable fonts, profiles, text reader, saturation, and page structure view where implemented.

Implementation prompt (continued)

Validation:
- Start the Vite dev server.
- Open each main route directly and through navigation.
- Run a sample analysis and open history.
- Confirm console errors are not introduced.

Acceptance criteria:
- CogniLens runs as a Vite + React frontend.
- Existing workflows remain usable.
- Local backend communication works.
- Accessibility widget features remain available.

After implementation, summarize:
- files changed
- React routes/components added
- legacy modules preserved
- proxy/routing fixes
- validation results

6. Eye evidence, history detail, heatmaps, and print/download modularisation

Commit evidence: 898f4c2, 2d2d939, 8f6e70e, 927055f, 5346dcc, fade214, 109411d, 54ad391, c255bcd

Representative contributors: YanyaL, runhong

What changed

Eye tracking became a more meaningful evidence layer. Heatmaps, element-hit logic, history summaries, detail panels, and print/download modularisation were added or refined.Added temporary HTML upload, scroll-synced Visited Map, and element-hit detection for gaze evidence.Replaced percentage-only eye evidence with risk-index summaries and heatmap detail panels in History.Split React print/download and dashboard interaction logic into more maintainable modules.

AI-assisted programming prompt

Implementation prompt

Upgrade the eye-tracking evidence workflow so history reports show meaningful behavioural evidence instead of raw percentages only.

Current context:
- Eye tracking can collect gaze samples and heatmap coverage.
- History can store and reopen analysis runs.
- The old evidence display is too numeric and does not explain what the user looked at.
- Do not build a full manual AOI annotation system.

Goal:
Connect gaze samples to page elements, calculate element-level evidence, and display concise interpretation in History and Heatmap detail views.

Part 1: Eye data capture and heatmap support
- Keep loading URL and temporary HTML workflows.
- Add or preserve scroll-synced heatmap/visited-map rendering.
- Make sure saved sessions include enough metadata for later review.

Implementation prompt (continued)

Part 2: Element-hit evidence
- Use DOM-based element detection to classify gaze targets as headings, interactive elements, main text, media, navigation, or other.
- Calculate hit count, dwell time, first fixation, share, and weighted share where available.
- Convert element evidence into Low / Medium / High risk labels.

Part 3: History and heatmap detail display
- Keep the History table compact.
- Show Eye Evidence availability and a short interpretation.
- Add a Heatmap detail panel with overall risk, confidence, interpretation, and risk drivers.

Part 4: Print/download modularisation
- Move print and download logic out of oversized dashboard files where appropriate.
- Preserve existing report export behaviour.

Implementation prompt (continued)

Validation:
- Run an eye session and save it.
- Open History and view the heatmap.
- Confirm element risk drivers render without crashing older records.
- Confirm print/download still works.

Acceptance criteria:
- Eye Evidence is explainable and connected to elements.
- History shows concise evidence summaries.
- Heatmap detail shows richer risk drivers.
- Print/download code is easier to maintain.

After implementation, summarize:
- files changed
- element-hit fields added
- scoring/display changes
- print/download refactor
- validation results

7. Detector-based refactor, dashboard stability, Priority Lens, and issue-card cleanup

Commit evidence: 0043724, 8729104, c531fbc, af739b0, 84e6f0d, cd3bde0, f52210c, a819632, e29b56a

Representative contributors: gldwen, Henrrryuan, YanyaL

What changed

This phase stabilised the product after major migrations. Detector selectors, dashboard state layers, Priority Lens behaviour, issue-card filtering, and history bugs were cleaned up.Refactored detector logic and patient/profile dashboard behaviour.Split dashboard concerns into state, actions, rendering, runtime, architecture, and observability layers.Fixed Priority Lens gauge behaviour, Top Issue expansion, zero-issue cards, and a History 500 error.

AI-assisted programming prompt

Implementation prompt

Stabilise the CogniLens detector and dashboard architecture after the React migration.

Current context:
- The project now has many detectors, React pages, legacy dashboard modules, Priority Lens behaviour, and history records.
- Some dashboard behaviour is unstable after recent refactors.
- Detector grounding and issue cards need to be more reliable.
- Do not change the product scope or rewrite unrelated pages.

Goal:
Improve detector organisation, dashboard state handling, Priority Lens behaviour, and issue-card reliability.

Part 1: Detector refactor
- Organise detector logic for dense text, language complexity, sentence complexity, long content, heading structure, navigation complexity, weak information prominence, visual overload, auto-moving content, and excessive interruptions.
- Improve selector/DOM grounding so detected issues can point to meaningful elements.
- Remove ghost issue cards caused by stale or empty data.

Implementation prompt (continued)

Part 2: Dashboard architecture
- Separate dashboard state, actions, rendering, runtime helpers, architecture constraints, and observability utilities.
- Preserve existing dashboard features while reducing reference errors.
- Keep module boundaries understandable for future contributors.

Part 3: Priority Lens and issue cards
- Fix Priority Lens gauge updates when profile changes.
- Expand relevant Top Issue cards correctly.
- Hide zero-issue cards where appropriate.
- Keep cognitive support labels clear.

Part 4: Bug fixes
- Investigate and fix History 500 errors related to new data shapes.
- Ensure old reports still load with safe defaults.

Implementation prompt (continued)

Validation:
- Run sample analyses across several detector categories.
- Change Priority Lens profile and confirm dashboard updates.
- Open History records and confirm no 500 error.
- Confirm zero-issue cards do not clutter the UI.

Acceptance criteria:
- Dashboard modules are more stable and maintainable.
- Detector output is better grounded to elements.
- Priority Lens and issue cards behave consistently.
- History remains compatible with previous records.

After implementation, summarize:
- files changed
- detector refactor details
- dashboard architecture changes
- bugs fixed
- validation results

8. Visual Complexity / ViCRAM added to the main product workflow

Commit evidence: cc20357, 45c584f, 4d6faf1, 5bb6170, 6e74d66, db9641f, b168961, c4db979

Representative contributors: gldwen, Henrrryuan, Dylan(yanya) liu, runhong

What changed

Visual complexity became a visible product module rather than a side experiment. It was connected to dashboard, loading, history, and persisted evidence.Added ViCRAM visual complexity routes, score display, grid generation, and dashboard panels.Added sidebar switching between Visual Complexity and Accessibility Findings.Persisted visual complexity per analysis run and split History evidence into Visual Complexity and Eye Evidence columns.

AI-assisted programming prompt

Implementation prompt

Add ViCRAM visual complexity analysis as a first-class CogniLens feature.

Current context:
- CogniLens already has rule-based cognitive accessibility findings and Eye Evidence.
- The project needs a separate visual complexity signal for page-level visual burden.
- Visual Complexity should not be mixed with Eye Evidence.
- Preserve dashboard and History behaviour for existing records.

Goal:
Integrate Visual Complexity into the main analysis workflow, dashboard, and history persistence.

Part 1: Backend visual complexity support
- Add or align routes/services for visual complexity analysis from HTML or URL input.
- Return page-level visual complexity score, risk label, page dimensions, text/image counts, grid data, and summary text where available.
- Keep results deterministic and explainable.

Implementation prompt (continued)

Part 2: Dashboard integration
- Add a Visual Complexity panel or tab in the dashboard.
- Display score, risk label, summary, and high-risk visual regions where available.
- Add sidebar switching between visual complexity and accessibility findings.

Part 3: History persistence
- Store visual complexity results per analysis run.
- Display Visual Complexity in its own History evidence column.
- Keep Eye Evidence separate.
- Use safe fallbacks for old records without visual complexity fields.

Part 4: Preview and rendering fixes
- Ensure visual complexity preview rendering works with uploaded or rendered pages.
- Fix range/state handling where dashboard data can be missing or delayed.

Implementation prompt (continued)

Validation:
- Analyse a visually simple page and a visually complex page.
- Confirm dashboard score and summary appear.
- Confirm History reloads persisted visual complexity results.
- Confirm Eye Evidence still displays separately.

Acceptance criteria:
- Visual Complexity is a first-class evidence type.
- Dashboard and History clearly separate Visual Complexity from Eye Evidence.
- Results persist across sessions.
- Missing visual complexity data does not crash old records.

After implementation, summarize:
- files changed
- backend fields/routes added
- dashboard display changes
- history persistence changes
- validation results

9. Final stabilisation: calibration, heatmap visibility, ZIP limits, and UI polish

Commit evidence: 484796c, 972d71b, 1e7172a, 542981f, a02e7d5, b350075, d707fd8, 13a7ce5

Representative contributors: runhong, Dylan(yanya) liu, Henrrryuan

What changed

The final phase focused on demo readiness: clearer calibration, less heatmap interference, manual heatmap visibility, onboarding updates, ZIP limits, and eye page styling cleanup.Improved eye-tracking calibration layout and hid calibration UI after tracking starts.Reduced heatmap interference and added manual heatmap visibility control.Updated onboarding, ZIP upload limits, and unified eye-tracking page styling.

AI-assisted programming prompt

Implementation prompt

Polish the final CogniLens prototype before submission and demonstration.

Current context:
- The main workflows are already implemented: analysis, dashboard, history, Eye Evidence, Visual Complexity, and ZIP/static site

support.
- The final demo needs a clearer and less distracting experience.
- Eye tracking controls and heatmap display can interfere with the tested webpage.
- Do not change core scoring or analysis logic unless required for stability.

Goal:
Improve final product usability, eye-tracking calibration, heatmap visibility, ZIP limits, onboarding, and UI consistency.

Part 1: Eye calibration polish
- Make the start/calibration CTA more visible.
- Improve calibration layout and spacing.
- Hide calibration UI after tracking starts so it does not block the tested webpage.

Implementation prompt (continued)

Part 2: Heatmap visibility control
- Reduce heatmap interference during active tracking.
- Show heatmap after pause or through a manual toggle.
- Keep Save Session and History heatmap review behaviour unchanged.

Part 3: Upload and onboarding polish
- Update ZIP size or validation limits where needed.
- Update onboarding guide steps to match current dashboard and eye-tracking flow.
- Simplify affected element text previews if they are too noisy.

Part 4: UI consistency
- Streamline the eye-tracking page design.
- Unify CSS and top-level styling with the rest of CogniLens.
- Remove or avoid unused controls that no longer support the workflow.

Implementation prompt (continued)

Validation:
- Start an eye-tracking session and confirm calibration UI disappears after tracking starts.
- Pause tracking and confirm heatmap visibility behaviour is clear.
- Upload or analyse a ZIP/static site within the updated limits.
- Open dashboard and history to confirm no regression.

Acceptance criteria:
- Eye tracking is clearer and less distracting during demos.
- Heatmap visibility is user-controlled or pause-based.
- ZIP upload guidance is consistent with product limits.
- Styling feels unified across the product.
- Existing analysis, history, and evidence features continue to work.

Implementation prompt (continued)

After implementation, summarize:
- files changed
- calibration/heatmap changes

- upload/onboarding changes
- UI polish changes
- validation results